

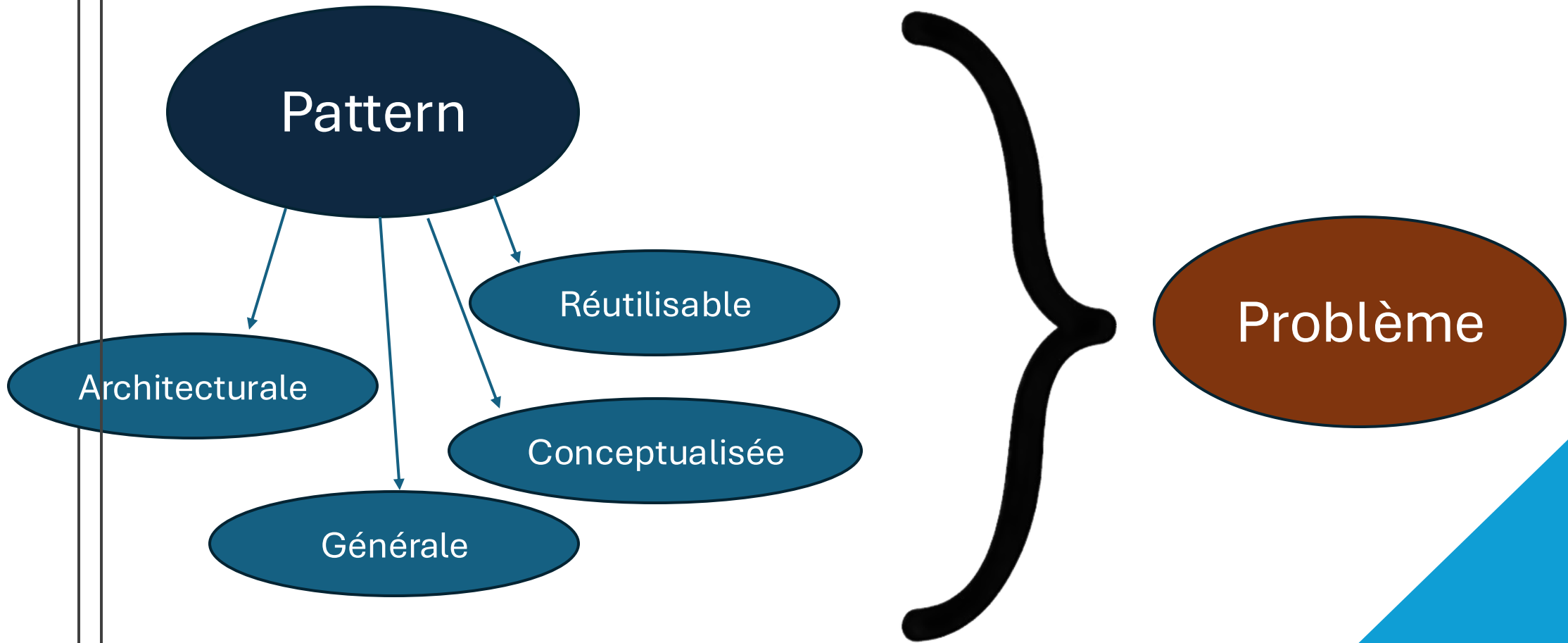
Pattern Strategie BUT INFO – R3.04

Evan ARRANGA
Adam ADNANE
Geoffrey RIGAL
Fatima ABOUDA

2026/2027



C'est quoi un pattern



Sommaire

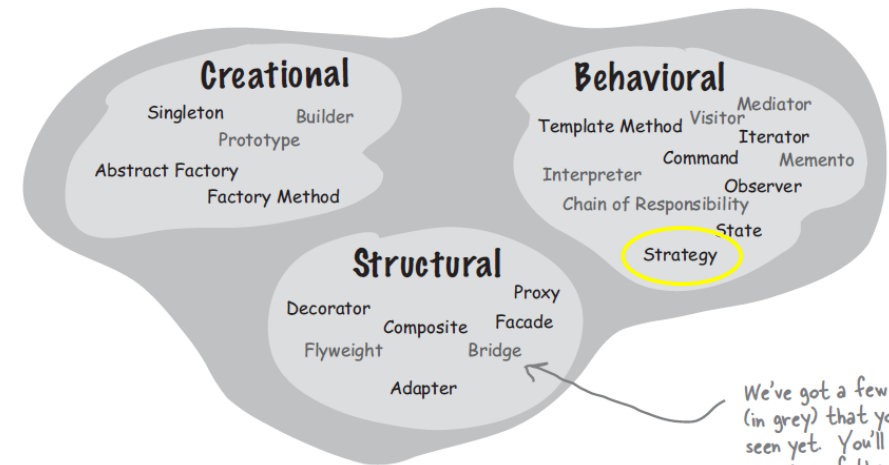
- Introduction GoF
- Problème concret
- Structure du pattern
- Lien avec SOLID
- Limites du pattern
- Bonnes pratiques
- Second exemple
- Exemples des canards
- Live coding
- QCM interactif

Intro GoF

- Pourquoi les patterns existent
- Les 3 familles (créationnels, structurels, comportementaux)
- Leur rôle dans la maintenabilité / extensibilité

from the objects it needs to
instantiate.

distribute responsibility.



Pattern Stratégie

Catégorie : Comportementale

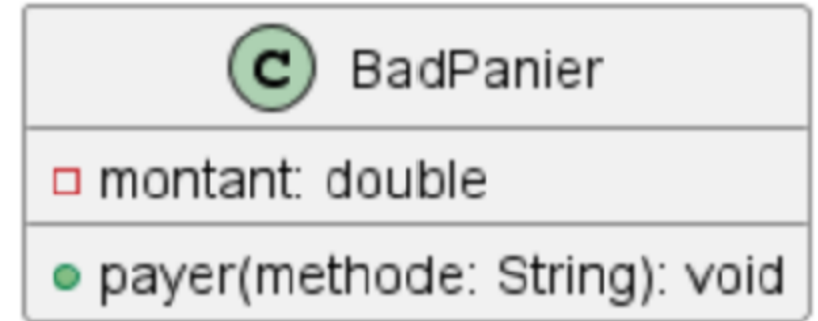
Séparer dans des classes distinctes

Algorithmes interchangeables

Réduit la complexité

Mauvaise modélisation

```
public class BadPanier {  
    private double montant;  
  
    public void payer(String methode) {  
        if (methode.equals("CB")) {  
            System.out.println("Validation du numéro de carte, date, cryptogramme...");  
            System.out.println("Paiement de " + montant + "€ par Carte Bancaire.");  
        } else if (methode.equals("PAYPAL")) {  
            System.out.println("Connexion à l'API PayPal, vérification de l'email...");  
            System.out.println("Paiement de " + montant + "€ via PayPal.");  
        } else if (methode.equals("VIREMENT")) {  
            System.out.println("Génération de l'IBAN et référence bancaire...");  
            System.out.println("En attente du virement de " + montant + "€.");  
        }  
    }  
}
```



Identifier problème

- Violation OCP
 - Ajout de méthodes de paiement : Modification de la classe
- Violation SRP
 - Responsable du moyen de paiement et du paiement
- Complexité
 - Arbre de If Else

```
package pattern_strategie_correct;

public class PaiementCarteBancaire implements StrategiePaiement {
    private String numeroCarte;
    private String cryptogramme;

    public PaiementCarteBancaire(String numeroCarte, String cryptogramme) {
        this.numeroCarte = numeroCarte;
        this.cryptogramme = cryptogramme;
    }

    @Override
    public void payer(double montant) {
        System.out.println(montant + "€ payés par Carte Bancaire (" + numeroCarte + ").");
    }
}
```

```
package pattern_strategie_correct;

public class PaiementPayPal implements StrategiePaiement {
    private String email;

    public PaiementPayPal(String email) {
        this.email = email;
    }

    @Override
    public void payer(double montant) {
        System.out.println(montant + "€ payés via le compte PayPal " + email + ".");
    }
}
```

Bonne modélisation

```
package pattern_strategie_correct;

public class PaiementVirement implements StrategiePaiement {
    private String NomCompte;
    private String RIB;

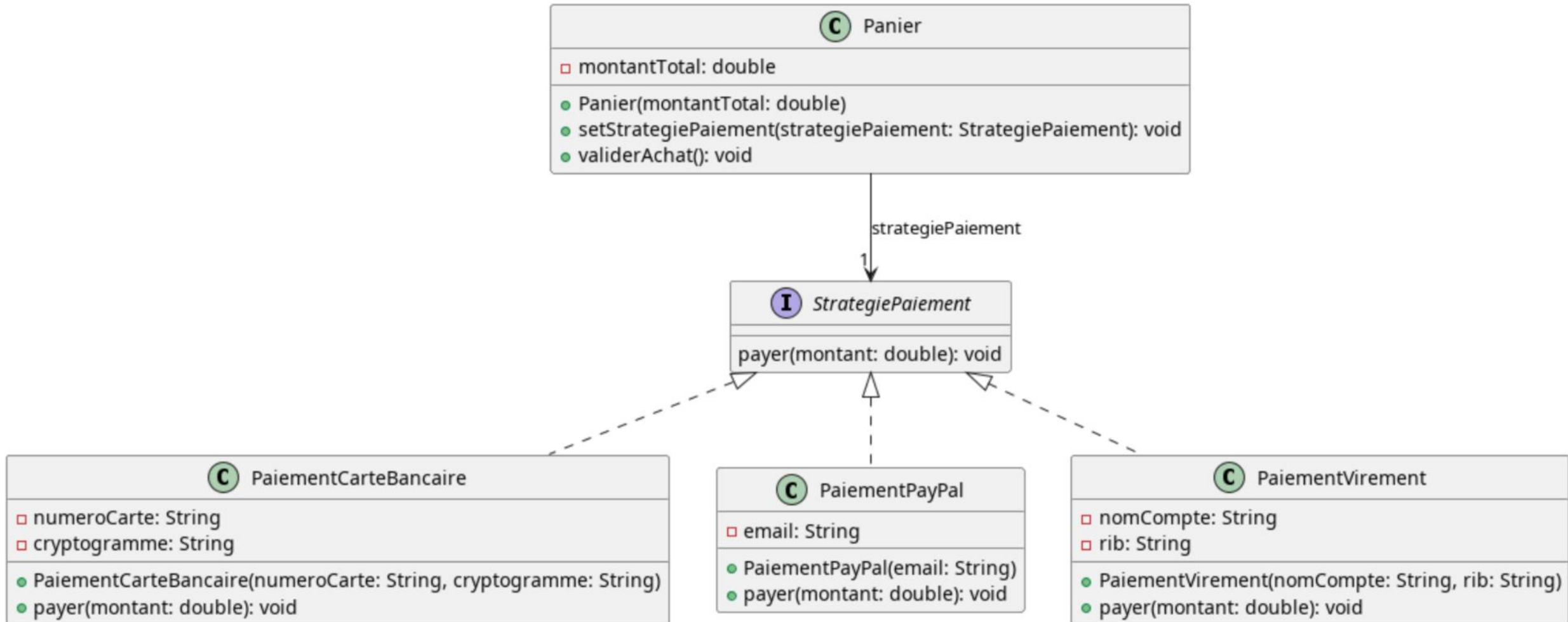
    public PaiementVirement(String NomCompte, String RIB) {
        this.NomCompte = NomCompte;
        this.RIB = RIB;
    }

    @Override
    public void payer(double montant) {
        System.out.println(montant + "€ payés par Virement vers : " + NomCompte + "(" + RIB +
        ").");
    }
}
```

Bonne modélisation

```
public class Panier {  
    private double montantTotal;  
    private StrategiePaiement strategiePaiement;  
  
    public Panier(double montantTotal) {  
        this.montantTotal = montantTotal;  
    }  
  
    public void setStrategiePaiement(StrategiePaiement strategiePaiement) {  
        this.strategiePaiement = strategiePaiement;  
    }  
  
    public void validerAchat() {  
        if (this.strategiePaiement == null) {  
            throw new IllegalStateException("Aucun moyen de paiement sélectionné !");  
        }  
        this.strategiePaiement.payer(this.montantTotal);  
    }  
}
```

Bonne modélisation

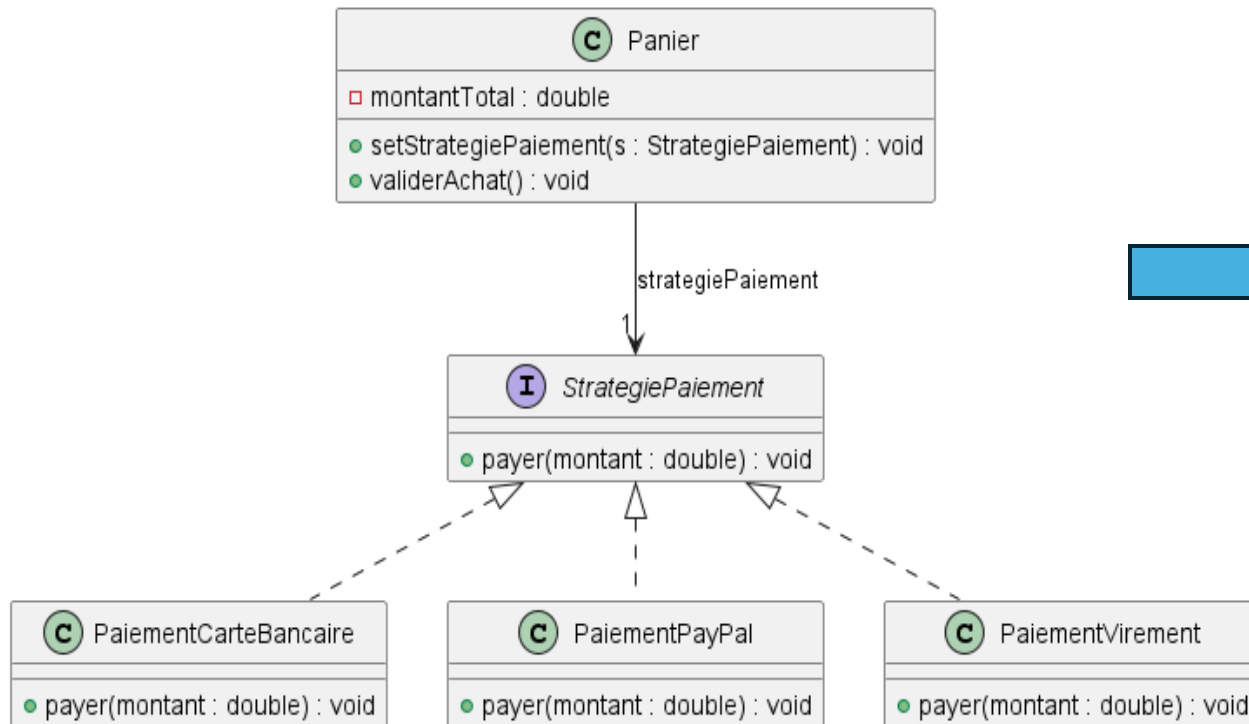


Bénéfices du pattern stratégie

- Élimination des branchements conditionnels
- Respect du principe ouvert/fermé (OCP)
- Interchangeabilité dynamique au runtime
- Isolation et testabilité unitaire
- Réutilisabilité du code

Du diagramme métier au diagramme générique

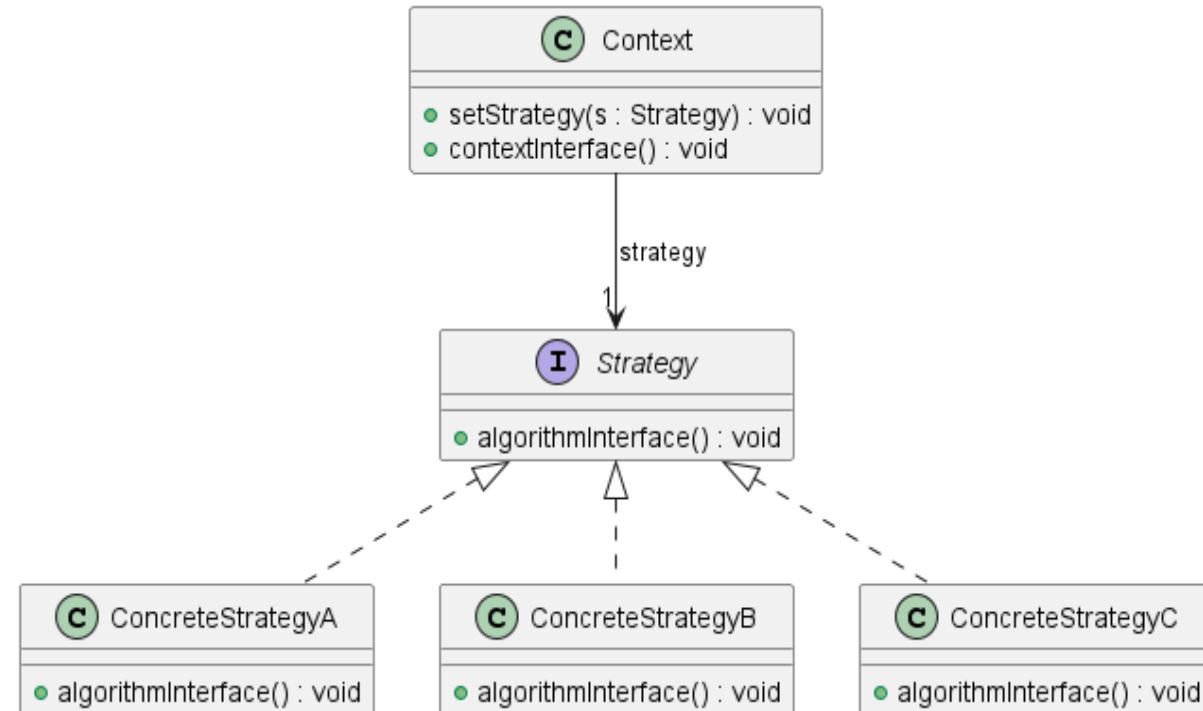
Exemple du paiement (diagramme métier)



Panier -> Context

StrategiePaiement -> Strategy

Pattern Stratégie (diagramme générique)



Paiement... -> ConcreteStrategy

Diagramme de classes du GoF (solution de la littérature)

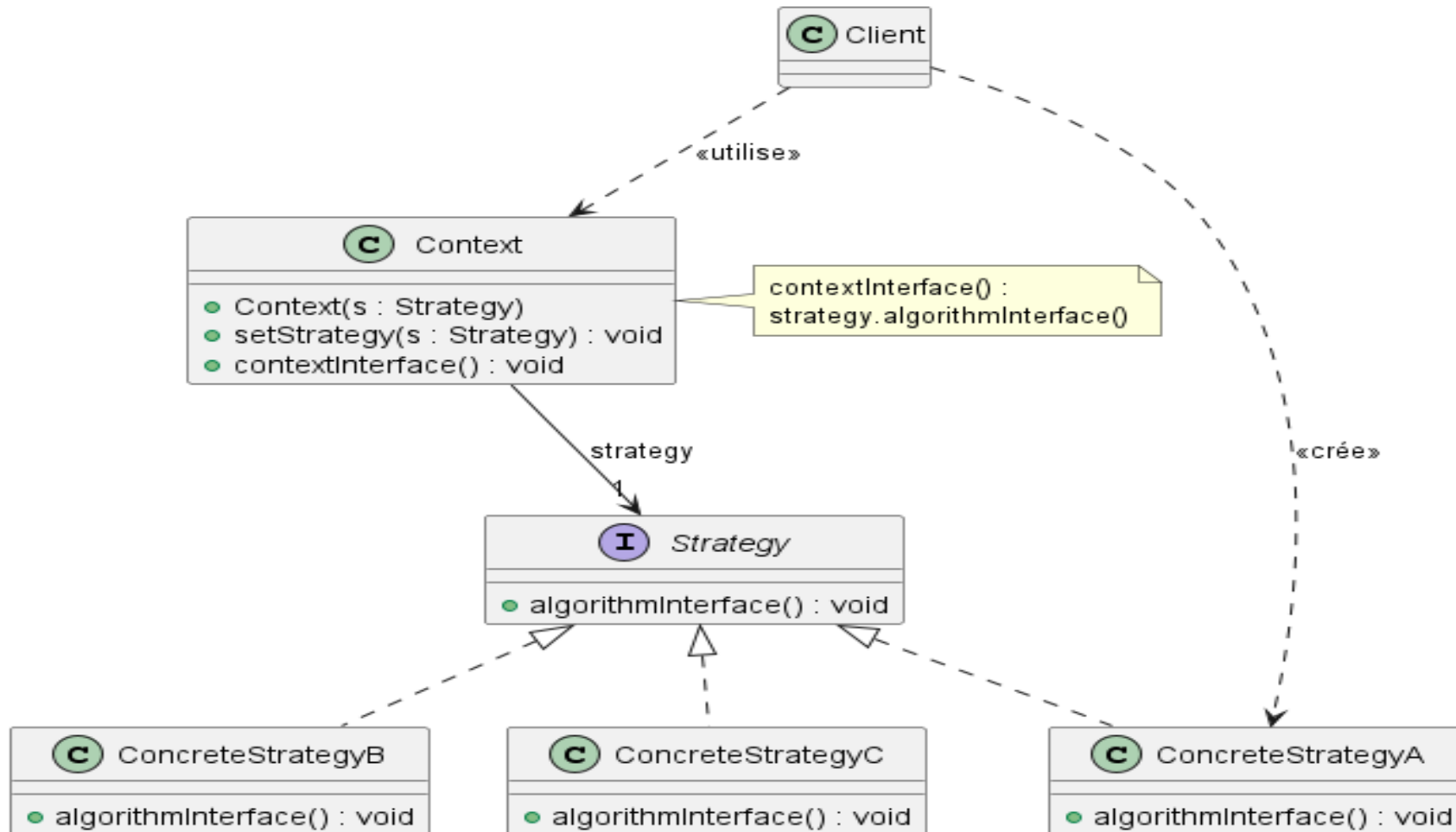
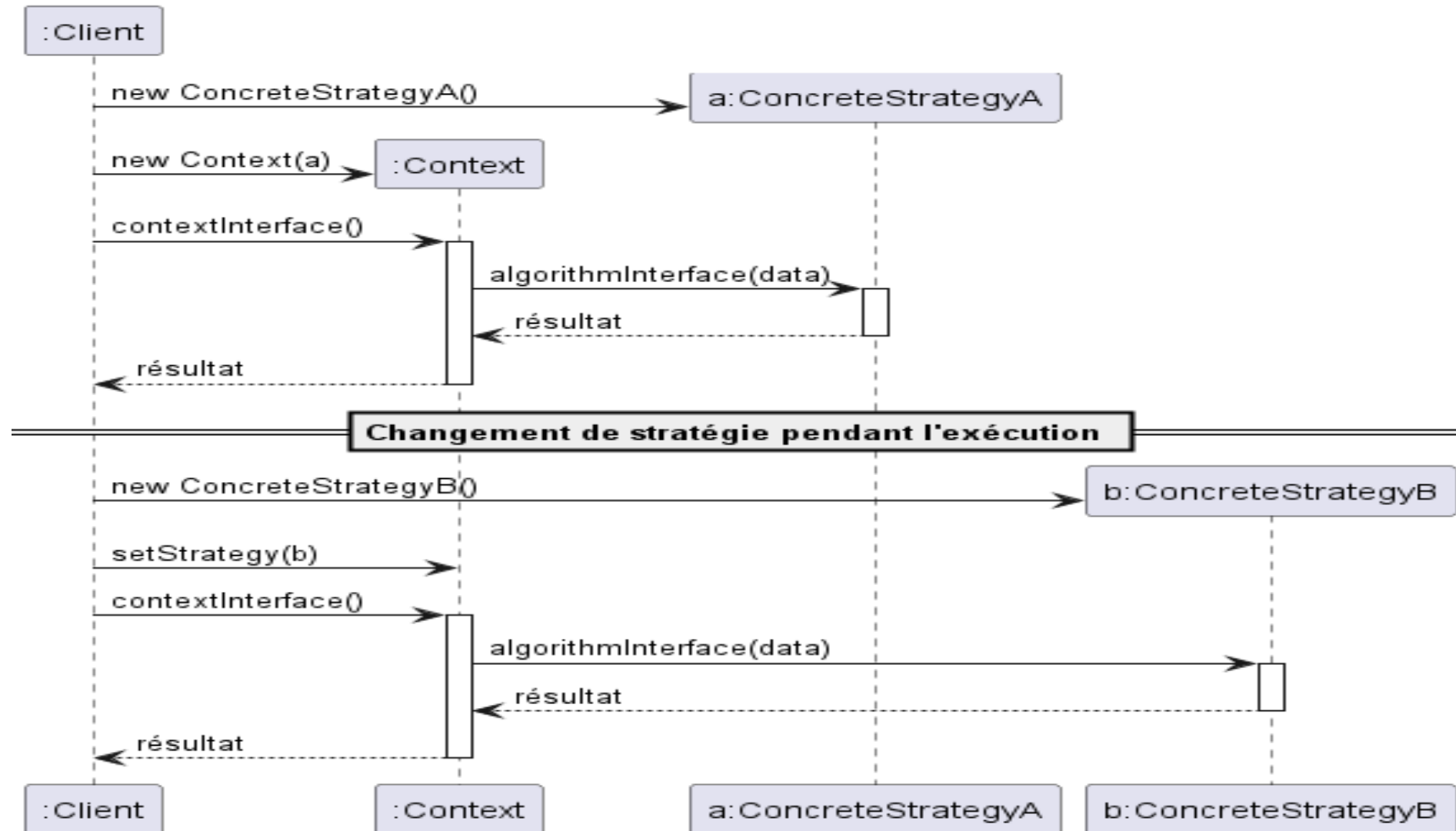


Diagramme de séquences du GoF



Les classes participantes

Participant	Rôle
Strategy (ex : StrategiePaiement)	Déclare l'interface commune à tout les algorithmes. Le contexte s'en sert pour appeler l'algorithme sans savoir lequel est utilisé.
ConcreteStrategy (ex : PaiementCarteBancaire, PaiementPaypal, PaiementVirement)	Implémente un algorithme précis en respectant l'interface Strategy, il y a une classe par façon de faire.
Context (ex : Panier)	Garde une référence vers une Strategy et lui délègue le travail. La stratégie peut être changé par un setter.
Client (ex : le main)	Choisit la ConcreteStrategy, la crée et la donne au contexte. Il peut la changer pendant l'exécution.

Lien avec SOLID

SRP : Single Responsibility Principle

OCP : Open/Close Principle

DIP : Dependency Inversion Principle

```

public class Panier {
    private double montantTotal;
    private StrategiePaiement strategiePaiement;

    public Panier(double montantTotal) {
        this.montantTotal = montantTotal;
    }

    public void setStrategiePaiement(StrategiePaiement strategiePaiement) {
        this.strategiePaiement = strategiePaiement;
    }

    public void validerAchat() {
        if (this.strategiePaiement == null) {
            throw new IllegalStateException("Aucun moyen de paiement sélectionné !");
        }
        this.strategiePaiement.payer(this.montantTotal);
    }
}

```

```

package pattern_strategie_correct;

public class PaiementPayPal implements StrategiePaiement {
    private String email;

    public PaiementPayPal(String email) {
        this.email = email;
    }

    @Override
    public void payer(double montant) {
        System.out.println(montant + "€ payés via le compte PayPal " + email + ".");
    }
}

```

```

package pattern_strategie_correct;

public class PaiementCarteBancaire implements StrategiePaiement {
    private String numeroCarte;
    private String cryptogramme;

    public PaiementCarteBancaire(String numeroCarte, String cryptogramme) {
        this.numeroCarte = numeroCarte;
        this.cryptogramme = cryptogramme;
    }

    @Override
    public void payer(double montant) {
        System.out.println(montant + "€ payés par Carte Bancaire (" + numeroCarte + ").");
    }
}

```

```

package pattern_strategie_correct;

public class PaiementVirement implements StrategiePaiement {
    private String NomCompte;
    private String RIB;

    public PaiementVirement(String NomCompte, String RIB) {
        this.NomCompte = NomCompte;
        this.RIB = RIB;
    }

    @Override
    public void payer(double montant) {
        System.out.println(montant + "€ payés par Virement vers : " + NomCompte + " (" + RIB + ").");
    }
}

```

Limites

- Multiplication du nombres classes
- Maintenance plus complexe
- Risque de sur-ingénierie
- Complexité initiale plus élevée

Bonne pratiques

- Interfaces petites et précises
- Implémentations sans état
- Définir une stratégie par défaut
- Séparer clairement la logique conditionnelle

Besoin métier

- Un site e-commerce doit appliquer différents types de réductions sur un prix :
 - réduction 10 %
 - réduction 20 %
- aucune réduction

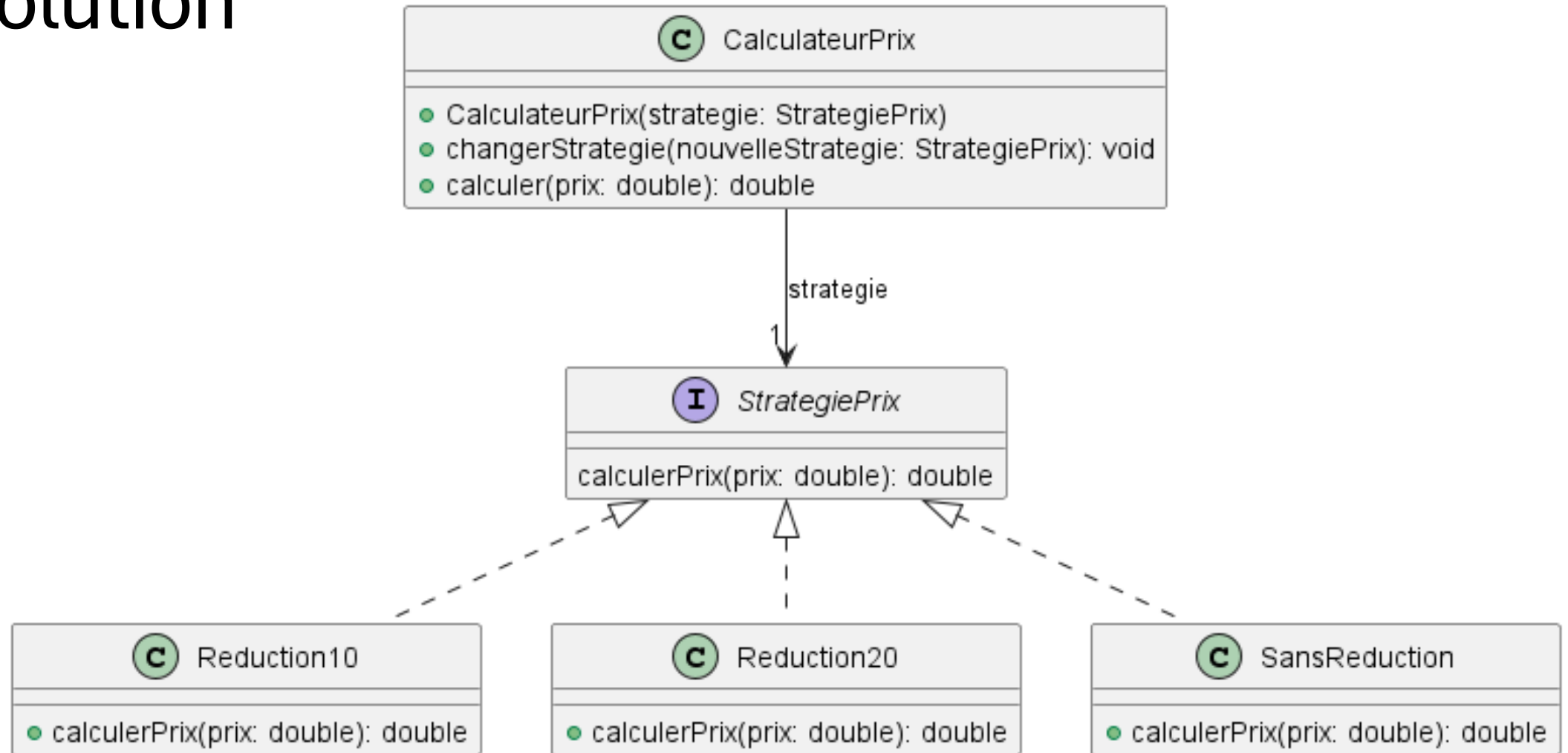
20

Mauvaise solution

- Ajout d'une nouvelle réduction = modification du code
- Multiplication des if
- Violations SRP, OCP

```
public double calculerPrix(double prix, String typeReduction) {  
    if (typeReduction.equals("AUCUNE")) {  
        return prix;  
    }  
    if (typeReduction.equals("REDUCTION10")) {  
        return prix * 0.90;  
    }  
    if (typeReduction.equals("REDUCTION20")) {  
        return prix * 0.80;  
    }  
    return prix;  
}
```

Solution



```
public interface StrategiePrix {  
    double calculerPrix(double prix);  
}  
  
public class Reduction10 implements StrategiePrix {  
    public double calculerPrix(double prix) { return prix * 0.9; }  
}  
  
public class CalculeurPrix {  
    private StrategiePrix strategie;  
    public double calculer(double prix) { return  
    strategie.calculerPrix(prix); }  
}
```

Code du
pattern
dans ce
contexte

23

```
public static void main(String[] args) {
```

```
    CalculeurPrix calculeur = new CalculeurPrix(new  
    SansReduction())
```

```
    System.out.println(calculeur.calculer(100));
```

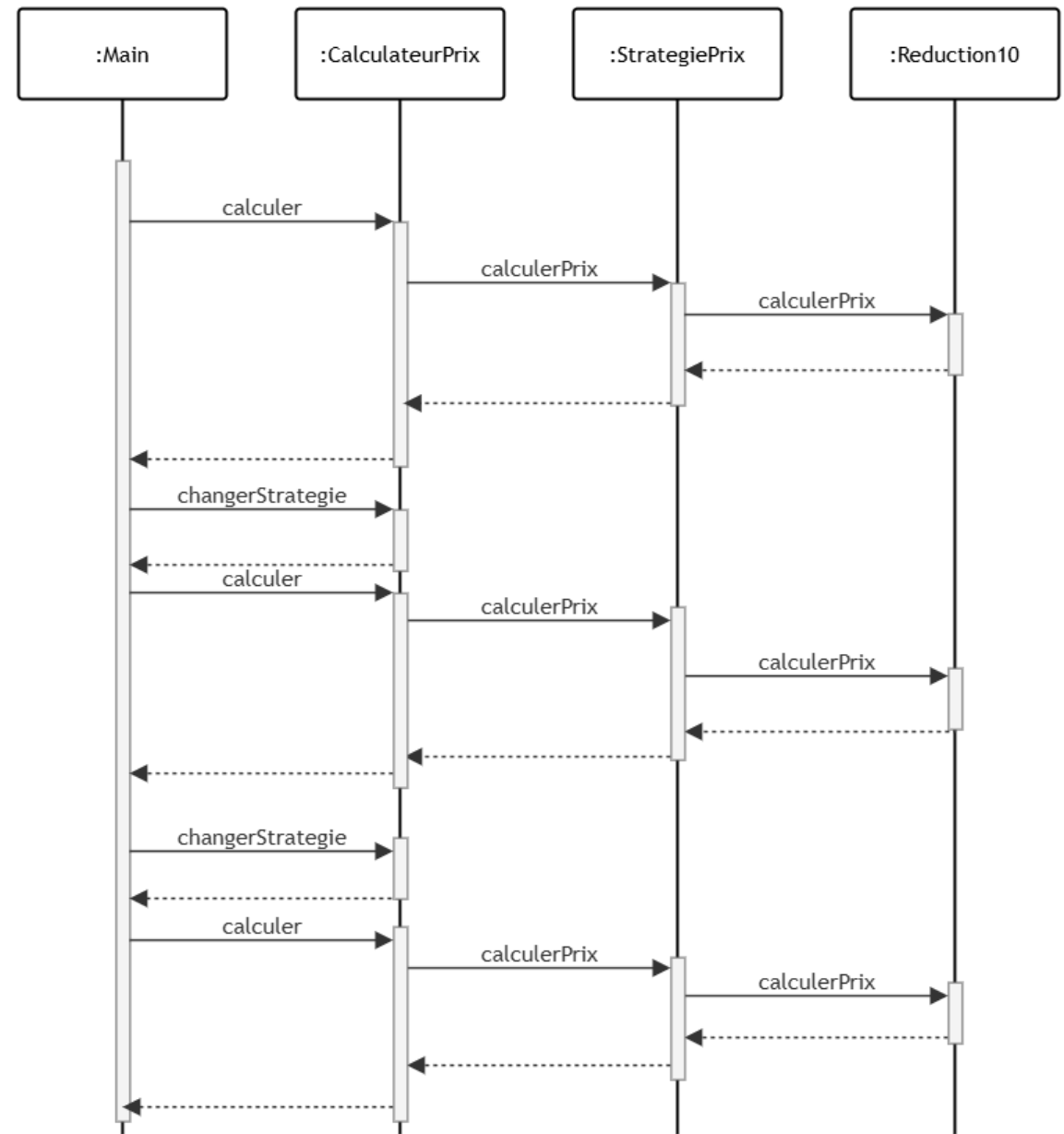
```
    calculeur.changerStrategie(new Reduction10());
```

```
    System.out.println(calculeur.calculer(100));
```

```
    calculeur.changerStrategie(new Reduction20());
```

```
    System.out.println(calculeur.calculer(100));
```

```
}
```

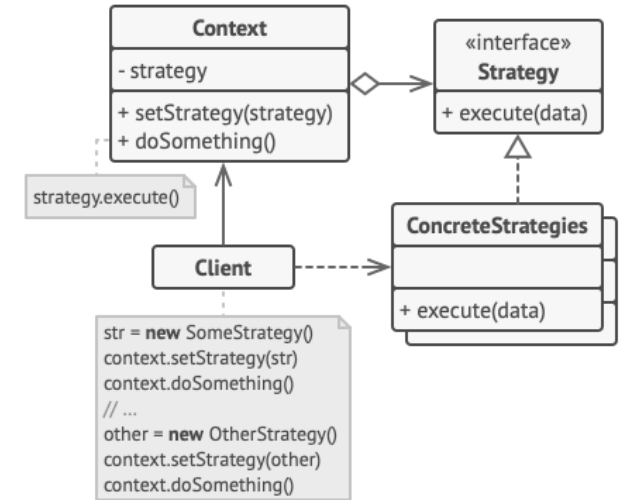
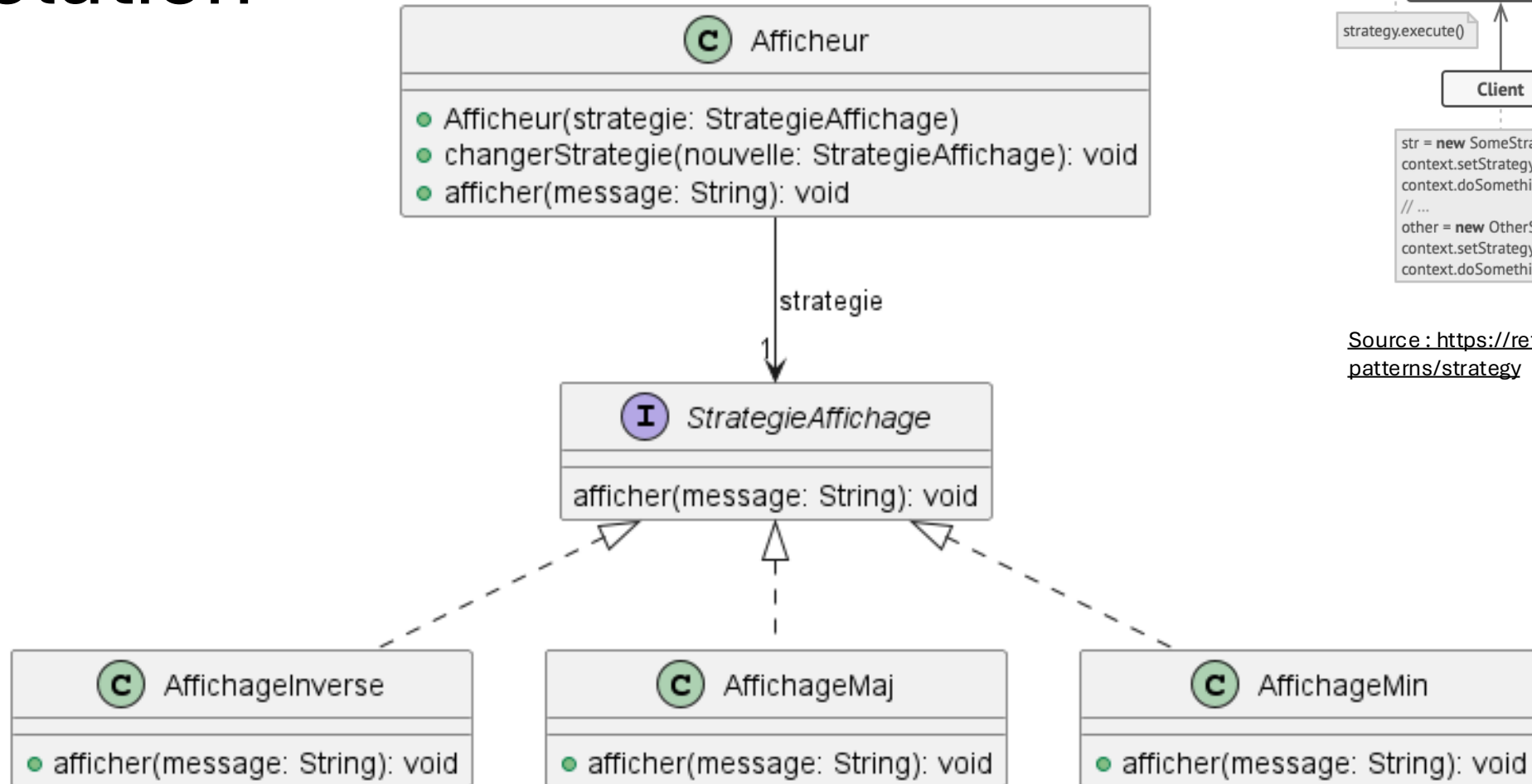


```
void afficher(String msg, String mode) {  
    if (mode.equals("maj")) System.out.println(msg.toUpperCase());  
    else if (mode.equals("min")) System.out.println(msg.toLowerCase());  
    else if (mode.equals("inv")) ...  
}
```

Besoin métier

- Afficher un message selon différents comportements :
 - majuscules
 - minuscules
 - inversé

Solution



Source : <https://refactoring.guru/fr/design-patterns/strategy>

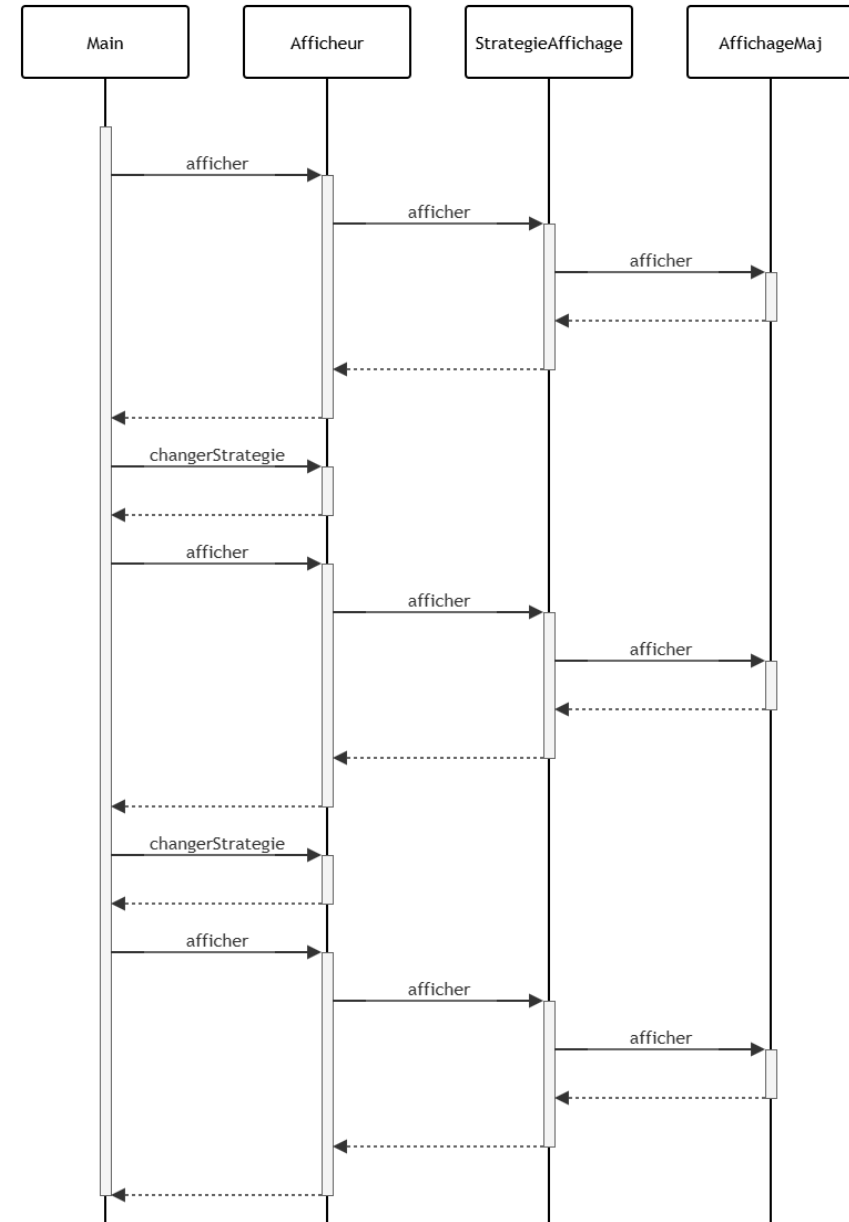
```

public static void main(String[] args) {
    Afficheur a = new Afficheur(new AffichageMaj());
    a.afficher("bonjour");

    a.changerStrategie(new AffichageMin());
    a.afficher("BONJOUR");

    a.changerStrategie(new AffichageInverse ());
    a.afficher("bonjour");
}

```

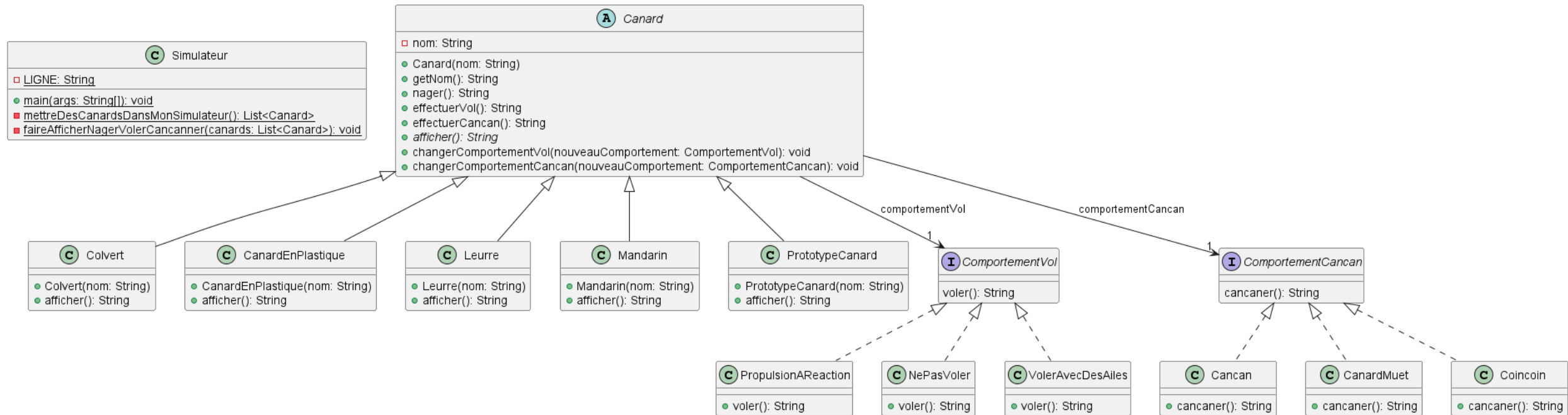


Besoin métier

- Les canards doivent :
 - voler différemment
 - cancaner différemment
 - parfois ne pas voler

Mauvaise solution

- La classe Canard contient tout :
 - voler()
 - cancaner()
- Violations SRP, OCP



Bonne Solution

```

public static void main(String[] args) {

    Canard c = new Colvert("colvert");

    System.out.println(c.afficher());

    System.out.println(c.effectuerVol());

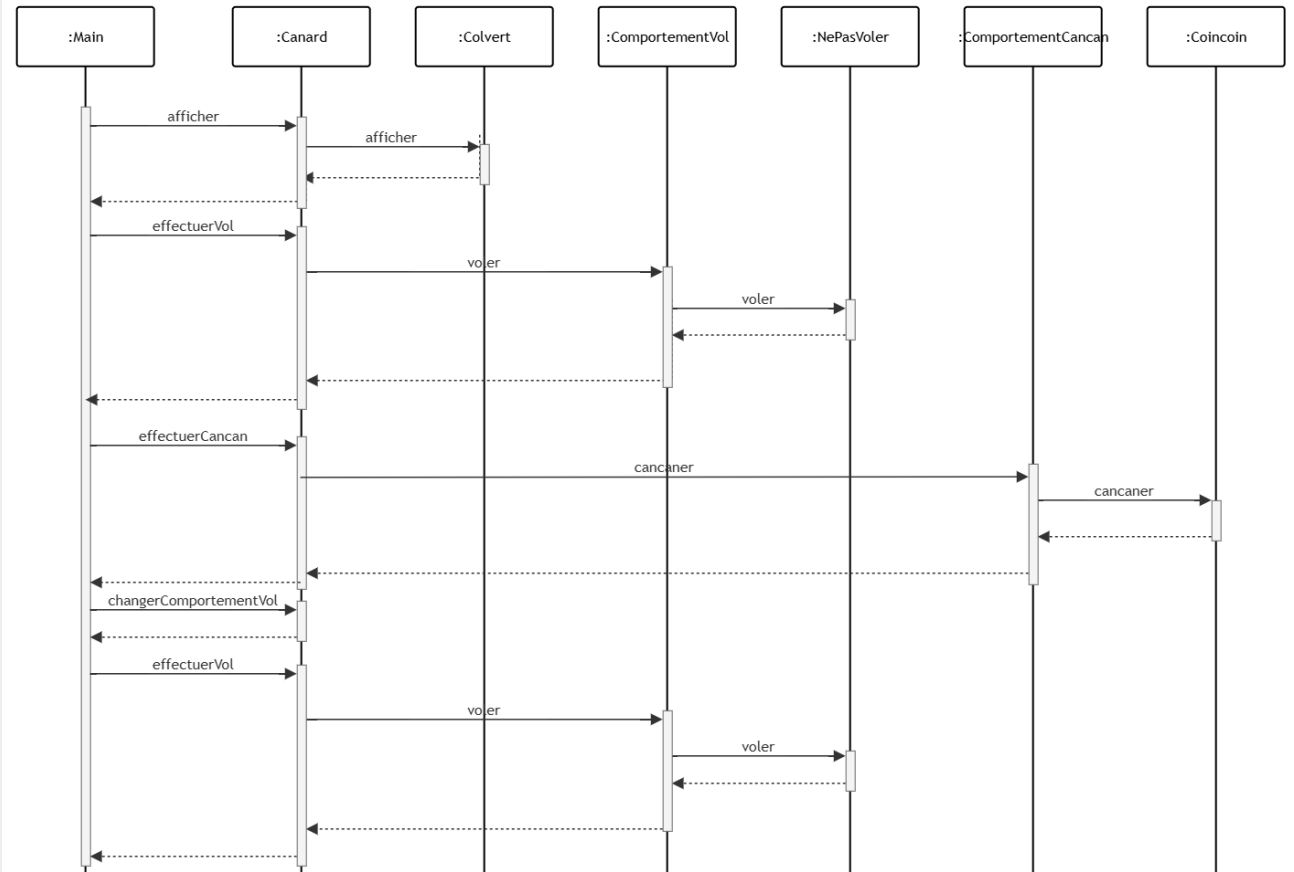
    System.out.println(c.effectuerCancan());

    c.changerComportementVol(new NePasVoler());

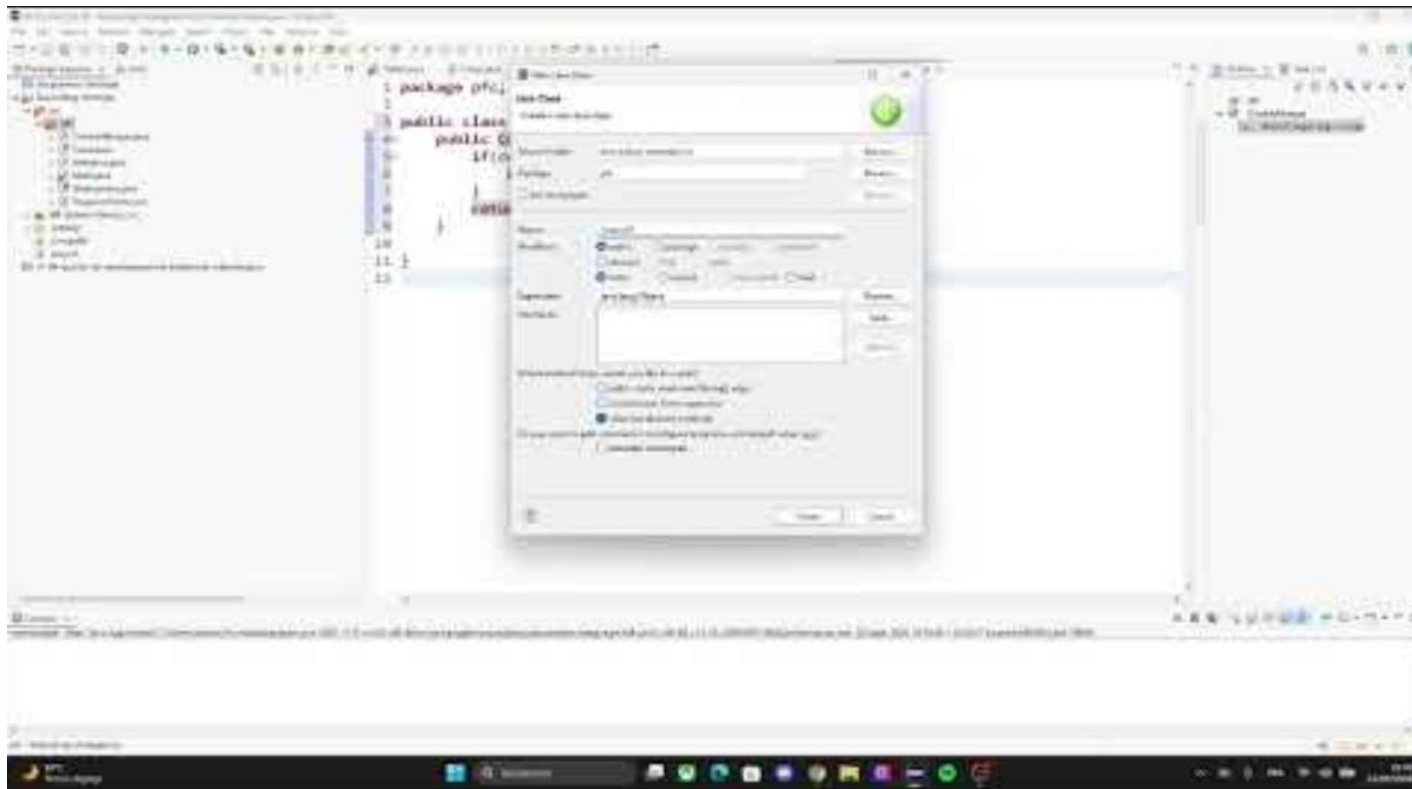
    System.out.println(c.effectuerVol());

}

```

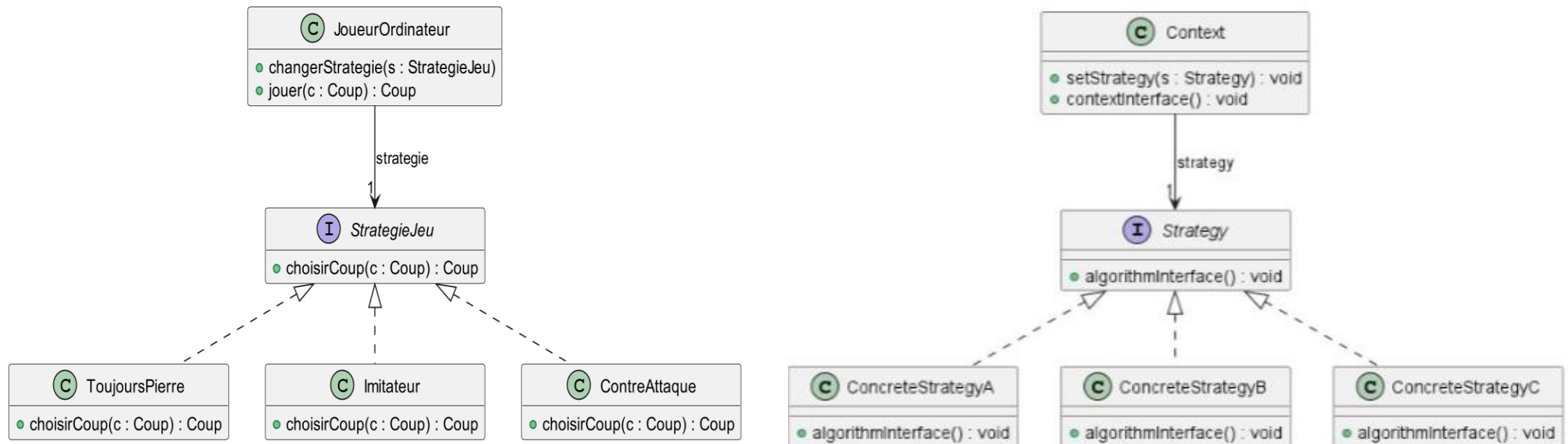


Live coding vidéo



33

Rétro conception du live coding





Quizz Pattern Stratégie

QCM

35

Bibliographie

- <https://stackify.com/strategy-pattern-definition-examples-and-best-practices/>
- Livre: Design Patterns Elements of Reusable Object-Oriented Software
- Livre: Les design patterns en Java
- <https://medium.com/@kethajayasandeep1254/strategy-pattern-java-design-patterns-6b756c712089>
- <https://java-design-patterns.com/patterns/strategy/>
- <https://www.youtube.com/watch?v=lWBmT22oCPk>
- <https://rpouiller.developpez.com/tutoriel/java/design-patterns-gang-of-four/?page=introduction>

IAGraphie

- Recherche pour un site de QCM "un outil de QCM pour 50 personnes gratuit dans le style kahoot" sur Copilot
- Vérification des exemple pour le Pattern Strategie "est ce que ça respecte le pattern Strategie" sur copilot
- Demande de transcription de code : "Peut tu réécrire le code en noir sur blanc de cette photos et des suivantes ?" (Gemini 3.8 Flash)
- Correction de fautes d'orthographe (Gemini 3.8 Flash)